Lecture 05.01: What is Object-Oriented Programming (OOP)?

📌 Executive Summary

Paradigm comparison between procedural and object-oriented programming, encapsulation of state and behavior, and modeling real-world domain entities.

🔬 Theoretical Foundations & Principles

1. Paradigm Shift: Procedural vs Object-Oriented

In procedural programming, code is organized around functions operating on passive data records:

$$\text{Program} = \text{Algorithms} + \text{Data Structures}$$

In Object-Oriented Programming (OOP), state (attributes) and behavior (methods) are unified into autonomous software entities called **objects**:

$$\text{Object} = \langle \text{State}, \text{Behavior}, \text{Identity} \rangle$$

2. The Four Pillars of OOP

1. **Encapsulation:** Bundling data and methods while restricting direct external access to internal state.
2. **Abstraction:** Exposing high-level essential interfaces while hiding low-level implementation mechanics.
3. **Inheritance:** Deriving new classes from existing ones to enable code reuse and hierarchical categorization.
4. **Polymorphism:** Allowing entities of different types to be treated through a unified common interface.

💻 Production Implementation & Code

```
# Conceptual contrast: Procedural vs OOP
# Procedural style
car_brand = "Tesla"
car_speed = 0

def accelerate(speed, increment):
    return speed + increment

# OOP style: State and behavior bundled together
class ElectricCar:
    def __init__(self, brand: str):
        self.brand = brand
        self.speed = 0

    def accelerate(self, increment: int) -> None:
        self.speed += increment
        print(f"{self.brand} accelerating to {self.speed} km/h")

my_car = ElectricCar("Tesla")
my_car.accelerate(40)
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.